03 Kontrollfluss

03 Kontrollfluss und Bedingungen

00 Übersicht · Zurück: 02f Arrays · Weiter: 03a Operatoren in Bedingungen

Vorlesung 19.10.2026
Themen: Programmablauf und Kontrollstrukturen · Formulierung und Auswertung von Bedingungen · Varianten der if-Anweisung (04 if-Anweisung)

Der Kontrollfluss

- Ein Programm besteht aus einer beliebigen Kombination von **einfachen** und **komplexen** Anweisungen (Kontrollstrukturen).
- **Kontrollfluss** = Ablauf des Programms, bestimmt durch die **Reihenfolge**, in der Anweisungen abgearbeitet werden. Bei der Programmierung legt man ihn fest.

Art	Beispiele
Einfache Anweisung	Wertzuweisung, Ausgabe auf der Konsole
Komplexe Anweisung	bedingte Anweisung (`if`), `while`-Schleife, `for`-Schleife, `switch`

Einfache Anweisungen

Werden **der Reihe nach (sequenziell)** vollständig abgearbeitet. Beispiele: Variable definieren, Wert zuweisen, initialisieren, einzelner Verarbeitungsschritt (z.B. Ausgabe mit `printf`).

```
initialisiere eine int Variable a mit einem Wert
initialisiere eine int Variable b mit einem Wert
addiere a zu b und speichere das Ergebnis in c
gib c auf der Console aus.
```

Bedingte Anweisung (`if`)

Eine Gruppe von Anweisungen wird **abhängig von einer Bedingung** ausgeführt → im Kontrollfluss entstehen **Verzweigungen**.

```
falls die Variable a den Wert 0 hat
  dann gib den Wert von b zurück
andernfalls
  berechne b - a und weise das Ergebnis der Variable a zu.
```

Schleifen

Eine Gruppe von Anweisungen wird so oft ausgeführt, bis eine Bedingung **nicht mehr** gilt.

```
solange b größer 0 ist
  ziehe a von b ab und weise das Ergebnis b zu.
```

Kombination von Kontrollstrukturen

- Anweisungen und Kontrollstrukturen können **beliebig kombiniert und geschachtelt** werden.
- Bedingte Anweisungen und Schleifen dürfen sich jedoch **nicht überschneiden**.
- Grenzen werden mit **geschweiften Klammern** `{ }` markiert. Sie sind Teil der Syntax und dürfen (i.d.R.) nicht weggelassen werden (**Compiler-Fehler**).
- Geschachtelte Strukturen werden durch **Einrückung** verdeutlicht (**Coding Convention**).

Beispiel (Pseudocode): Was wird berechnet?

```
initialisiere eine Variable a vom Typ int
initialisiere eine Variable b vom Typ int
falls a gleich 0 {
    dann gib b zurück
} andernfalls {
    solange b ungleich 0 {
        falls a größer b {
            weise a den Wert von a - b zu
        } andernfalls {
            weise b den Wert von b - a zu
        }
    }
}
gib a zurück
```

> **Auflösung**
> Das ist der **Euklidische Algorithmus** (Subtraktionsvariante): berechnet den **größten gemeinsamen Teiler (ggT)** von a und b. Der Fall `a == 0` liefert direkt `b`. (Die Folie verrät die Lösung nicht, dies ist meine Einordnung.)

Bedingungen im Kontrollfluss

- Eine **Bedingung** ist ein **Ausdruck**, dessen Auswertung immer einen **Wahrheitswert** (`true`/`false`) liefert.
- Eine Bedingung ist **keine Anweisung**, kann also nicht für sich allein stehen.
- Sie kann in einer einfachen Anweisung einer `boolean`-Variable zugewiesen werden:

```
int x = …;              // lies Wert für x von der Kommandozeile ein
boolean b = x > 0;
```

- Vor allem nötig für **bedingte Anweisungen** und **Schleifen**: Bedingung im `if` erlaubt unterschiedliche Abläufe; in der `while`-Schleife führt sie zur Beendigung, sobald sie nicht mehr wahr ist.

Operatoren zur Formulierung: 03a Operatoren in Bedingungen

Zusammenfassung

- Der Kontrollfluss entspricht dem **Algorithmus**, der umgesetzt werden soll; er wird über Kontrollstrukturen im Code nachgebildet.
- Unterscheide: einfache Anweisungen, bedingte Anweisungen, Schleifen und Switches.
- Der Ablauf wird in vielen Kontrollstrukturen über **Bedingungen** gesteuert.
- Zur Formulierung: **arithmetische, relationale und logische Operatoren**; die Regeln zur Auswertungsreihenfolge sind zu beachten.